

Deep Reinforcement Learning for Electric Vehicle Routing Optimization

Energy-Minimizing Electric Vehicle Routing Problem (EM-EVRP)

Dhia Elhak Ezzeddini Salem Fradi

Higher School of Communication of Tunis
Carthage University

Supervisor: Dr. Ferdaoues Chaabane
Host Company: Sagemcom

Academic Year 2025–2026

Outline

- 1 Introduction
- 2 Problem Statement
- 3 Mathematical Formulation
- 4 Baseline Optimization Methods
- 5 Deep Reinforcement Learning Approach
- 6 System Architecture
- 7 Results and Evaluation

Global EV Adoption Trends:

- Global EV sales exceeded **14 million** in 2023
- Expected to reach **40+ million** by 2030
- Major push from governments and corporations

Key Drivers:

- **Climate change** mitigation goals
- Stricter **emission regulations**
- Decreasing **battery costs**
- Government **incentives** and subsidies

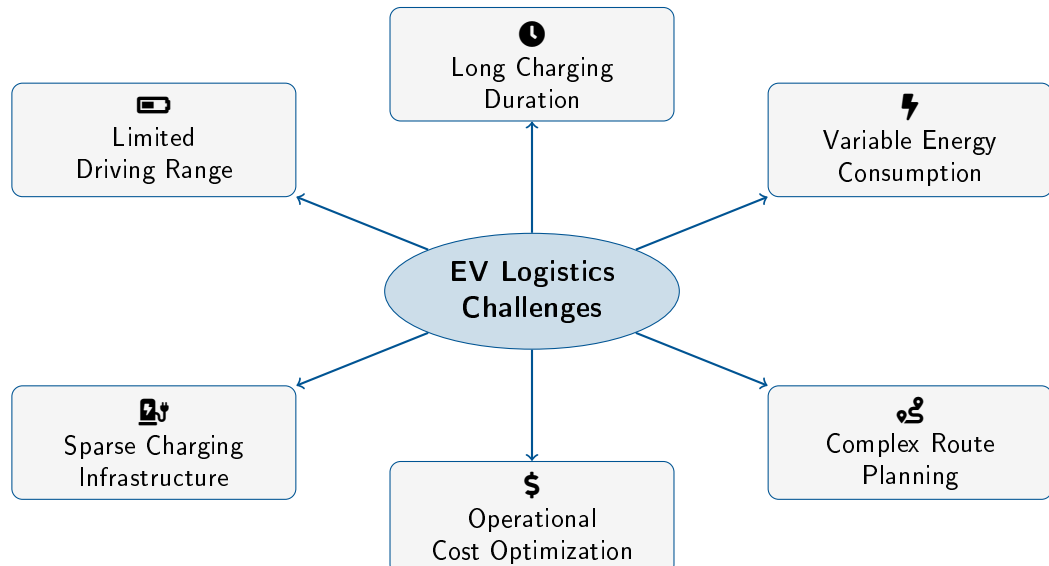
Why EVs for Logistics?

- Lower operating costs
- Zero tailpipe emissions
- Urban delivery restrictions
- Corporate sustainability

Challenge

Limited range & charging infrastructure require **smart routing**

Challenges in EV Logistics



Motivation: Why This Project?

Industry Needs

- Growing demand for **sustainable logistics**
- Need for **real-time** routing decisions
- Integration with **smart city** infrastructure
- **Cost reduction** through energy optimization

Technical Gaps

- Traditional VRP methods **don't scale**
- Energy models often **oversimplified**
- Limited **real-world** deployment tools
- Need for **learning-based** approaches

Our Contribution

An **end-to-end solution** combining Deep Reinforcement Learning with physics-based energy modeling and real-world deployment infrastructure

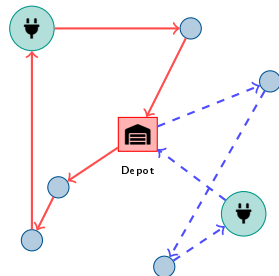
Electric Vehicle Routing Problem (EVRP)

Classical VRP:

- Serve all customers from depot
- Minimize total travel distance/cost
- Respect vehicle capacity

EVRP Extension:

- **Battery capacity** constraints
- **Charging station** visits allowed
- **Energy consumption** varies with load
- **Recharging time** impacts schedule



Multiple routes with charging stops

Energy-Minimizing EVRP (EM-EVRP)

Problem Definition

Objective: Minimize **total energy consumption** while serving all customers

Subject to Constraints:

① Customer Service

- Each customer visited exactly once
- Full demand satisfaction

② Vehicle Capacity

- Total load $\leq$ max capacity
- Partial deliveries possible

③ Battery SOC

- SOC ≥ 0 at all times
- Recharge at stations

④ Time Windows

- Service within $[a_i, b_i]$
- Route time budget

⑤ Route Feasibility

- Start and end at depot
- Valid path existence

⑥ Charging Logic

- Optional station visits
- Charging time cost

Why Minimize Energy Instead of Distance?

Distance $\neq$ Energy

Shortest path may not be most energy-efficient!

Energy depends on:

- **Vehicle load** (heavier = more energy)
- **Road gradient** (uphill = more energy)
- **Speed profile** (higher speed = more drag)
- **Regenerative braking** (downhill = energy recovery)

Energy Optimization Benefits

- **Extended range** per charge
- **Fewer charging** stops needed
- **Lower operational** costs
- **Reduced battery** degradation
- More **sustainable** operations

Key Insight: A longer route going downhill may consume

MILP Formulation: Sets and Parameters

Sets

N	All nodes (depot + customers)
C	Customers $C = N \setminus \{0\}$
S	Charging stations
K	Fleet of vehicles
R	Trip indices

Network Parameters

d_i	Demand at customer i
Q_k	Capacity of vehicle k
t_{ij}	Travel time on arc (i, j)
$[a_i, b_i]$	Time window at node i

EV Energy Parameters

m_c	Vehicle curb mass
C_d	Aerodynamic drag coeff.
C_r	Rolling resistance coeff.
ρ	Air density
A	Frontal area
α_{ij}	Slope angle on arc
$\varphi_{d/r}$	Motor efficiency
$\phi_{d/r}$	Battery efficiency

Physics-Based Energy Consumption Model

Total Vehicle Mass on Arc (i, j)

$$m_{ij} = m_c + u_{ij}$$

where u_{ij} is the current cargo load

Mechanical Power Required

$$P_{ij} = \underbrace{\frac{1}{2} C_d \rho A v_{ij}^2}_{\text{Aerodynamic drag}} + \underbrace{m_{ij} g \sin \alpha_{ij}}_{\text{Gravitational}} + \underbrace{m_{ij} g C_r \cos \alpha_{ij}}_{\text{Rolling resistance}}$$

Energy Consumption with Efficiency

$$E_{ij} = \begin{cases} \phi_d \cdot \varphi_d \cdot P_{ij} \cdot t_{ij} & \text{if } P_{ij} \geq 0 \text{ (propulsion)} \\ \phi_r \cdot \varphi_r \cdot P_{ij} \cdot t_{ij} & \text{if } P_{ij} < 0 \text{ (regeneration)} \end{cases}$$

MILP Formulation: Objective and Constraints

Objective Function

$$\min \sum_{k \in K} \sum_{r \in R} \sum_{(i,j) \in A} E_{ij} \cdot x_{ijk}^r$$

Minimize total energy consumption across all routes

Customer Service:

$$\sum_{k \in K} \sum_{r \in R} y_{ik}^r = 1, \quad \forall i \in C$$

Capacity:

$$\sum_{i \in C} d_i \cdot y_{ik}^r \leq Q_k, \quad \forall k, r$$

Flow Conservation:

$$\sum_j x_{ijk}^r = y_{ik}^r = \sum_j x_{jik}^r$$

Subtour Elimination (MTZ):

$$w_{ik}^r - w_{jk}^r + |C| x_{ijk}^r \leq |C| - 1$$

Classical Optimization Approaches

We implemented **four baseline methods** for comparison:

1. Google OR-Tools

- Industry-standard solver
- Constraint programming
- Arc cost callbacks for energy
- Local search improvements

2. Hill Climbing

- Simple local search
- 2-opt, Or-opt, Exchange
- Fast but local optima

3. Simulated Annealing

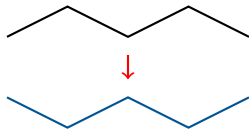
- Probabilistic acceptance
- Escapes local optima
- Temperature-based control
- $P(\text{accept}) = e^{-\Delta E/T}$

4. Ant Colony Optimization

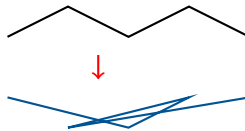
- Nature-inspired
- Pheromone trails
- Collective intelligence
- Good for combinatorial

Metaheuristics: Local Search Operators

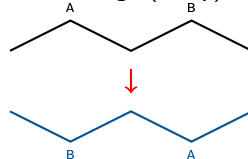
2-Opt (Reverse Segment)



Or-Opt (Relocate)



Exchange (Swap)



Simulated Annealing Acceptance Rule

$$P(\text{accept worse solution}) = \exp\left(-\frac{\Delta E}{T}\right)$$

- High temperature $\Rightarrow$ explore (accept worse solutions)
- Low temperature $\Rightarrow$ exploit (only accept improvements)
- Cooling schedule: $T_{k+1} = \alpha \cdot T_k$ where $\alpha \in [0.9, 0.99]$

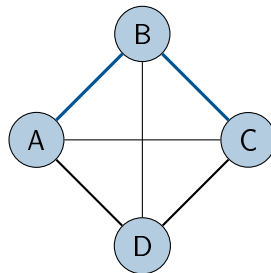
Ant Colony Optimization

Probabilistic Transition Rule:

$$p_{ij}^a = \frac{[\tau_{ij}]^\alpha \cdot [\eta_{ij}]^\beta}{\sum_{l \in \mathcal{N}_i} [\tau_{il}]^\alpha \cdot [\eta_{il}]^\beta}$$

where:

- τ_{ij} = pheromone level on arc (i, j)
- $\eta_{ij} = 1/c_{ij}$ = heuristic information
- α, β = importance parameters



Line thickness = pheromone level

Pheromone Update:

$$\tau_{ij} \leftarrow (1 - \rho)\tau_{ij} + \sum_a \Delta\tau_{ij}^a$$

Better solutions deposit more pheromone

Key Properties

- Positive feedback
- Parallel exploration
- Emergent optimization

Why Deep Reinforcement Learning?

Traditional Methods Limitations

- Exact methods: exponential time
- Heuristics: problem-specific design
- No generalization to new instances
- Slow for real-time applications

DRL Advantages

- **Learn** to solve, not program
- **Generalize** to new instances
- **Fast inference** (milliseconds)
- **End-to-end** optimization

Key Insight

Train a neural network to **construct** solutions step-by-step, learning from experience which decisions lead to lower energy consumption

Inspired by: Kool et al., “Attention, Learn to Solve Routing Problems!” (ICLR 2019)

Markov Decision Process Components

State s_t : Current partial solution + vehicle status

- Visited nodes, current position
- Remaining capacity, SOC, time budget

Action a_t : Select next node to visit

- Customer, charging station, or depot

Reward r_t : Negative energy for traversed arc

- Minimize cumulative energy consumption

Policy π_θ : Neural network with parameters θ

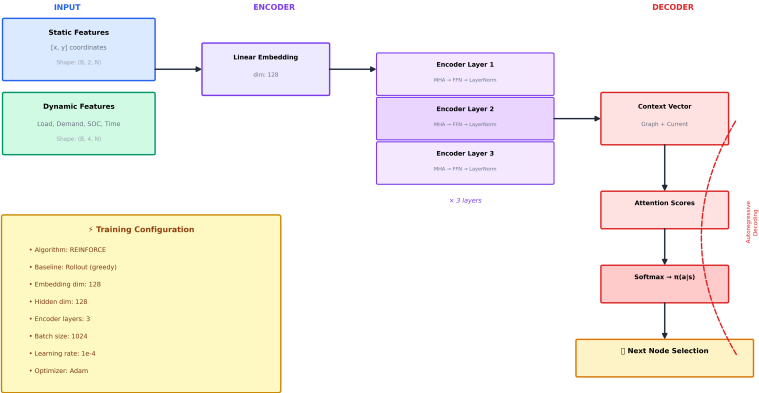
- Maps states to action probabilities

$$\textbf{Objective: } \max_{\theta} \mathbb{E}_{\pi_{\theta}} [\sum_t r_t] = \max_{\theta} \mathbb{E}_{\pi_{\theta}} [-\sum_t E_t]$$

Attention-Based Architecture: Overview

□ Attention-Based DRL Model Architecture

Encoder-Decoder with Multi-Head Attention for EVRP



Multi-Head Self-Attention Layer

$$h_i^{(l)} = \text{BN} \left(h_i^{(l-1)} + \text{MHA} \left(h_i^{(l-1)}, H^{(l-1)} \right) \right)$$

Attention Mechanism:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Graph Embedding:

$$\bar{h} = \frac{1}{n} \sum_{i=1}^n h_i$$

Mean aggregation of all node embeddings

Architecture Details

Embedding dim	128
Attention heads	8
Encoder layers	3
FF hidden	512
Normalization	Batch

Context-Aware Decoder

Context Embedding

$$c_t = W_c [\bar{h}; h_{\pi_{t-1}}; \text{load}_t; \text{SOC}_t; \text{time}_t]$$

Combines: graph embedding + previous node + vehicle state

Attention-Based Selection

$$u_{tj} = \begin{cases} C \cdot \tanh\left(\frac{q_t \cdot k_j}{\sqrt{d_k}}\right) & \text{if } j \in \mathcal{A}_t \\ -\infty & \text{otherwise} \end{cases}$$

$$p_{\theta}(a_t = j | s_t) = \text{softmax}(u_t)_j$$

Feasibility Masking

$\mathcal{A}_t$ = nodes that satisfy: demand $\leq$ capacity AND energy needed $\leq$ SOC

REINFORCE Training Algorithm

Policy Gradient Objective

$$J(\theta) = \mathbb{E}_{\pi_{\theta}} [L(\pi)]$$

where $L(\pi)$ is the total route energy consumption

Gradient Estimation with Baseline

$$\nabla_{\theta} J \approx \frac{1}{B} \sum_{i=1}^B (L(\pi_i) - b(s_i)) \nabla_{\theta} \log p_{\theta}(\pi_i | s_i)$$

- $b(s)$ = baseline value (reduces variance)
- We use **rollout baseline**: greedy policy copy

Baseline Update Rule

Update baseline when current policy shows statistically significant improvement

State of Charge Tracking

$$\text{SOC}_{t+1} = \text{SOC}_t - \frac{E_{ij}}{E_{\text{battery}}}$$

- Continuous SOC monitoring
- Reset at charging stations
- Negative consumption = regeneration

Load Dynamics

$$\text{load}_{t+1} = \text{load}_t - d_{\text{served}}$$

Cargo decreases after each delivery

Dynamic Masking Rules

- 1 Sufficient SOC for arc?
- 2 Can reach depot from there?
- 3 Enough capacity for demand?
- 4 Time budget remaining?
- 5 Station detour feasible?

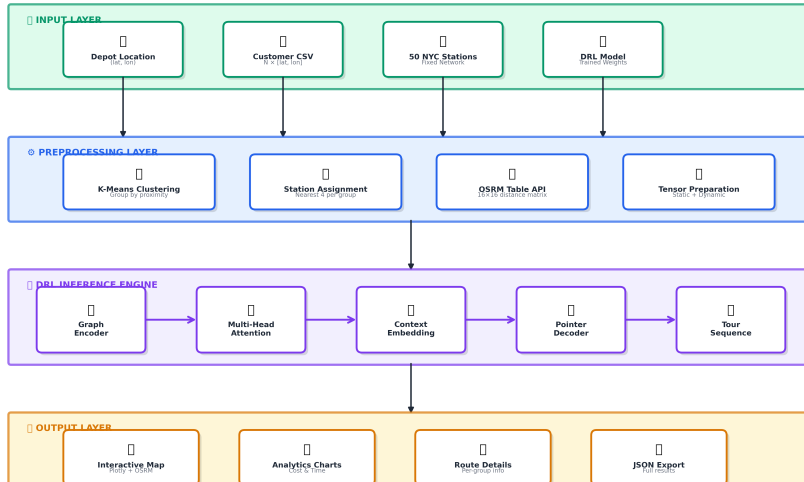
Learning Charging Decisions

Model learns **when** to visit charging stations through experience

End-to-End System Pipeline

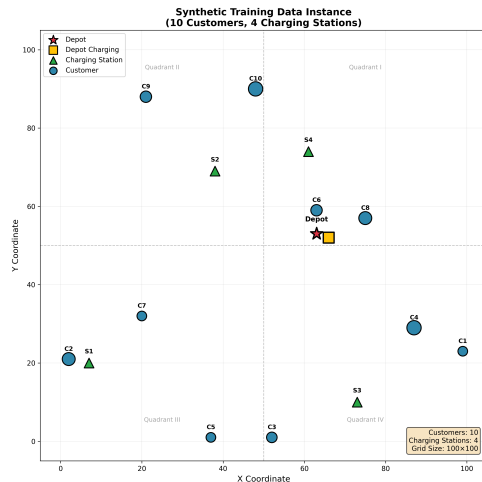
⚡ EV Route Optimization System Architecture

Deep Reinforcement Learning with Real Street Routing



Training Data Generation:

- 1 Random coordinates for nodes
 - Depot: $[25,75] \times [25,75]$
 - Customers: $[0,101] \times [0,101]$
 - Stations: quadrant-based placement
- 2 Random elevations $[0,101]$
 - Normalized for slope calculation
- 3 Random demands $[1, \text{max_demand}]$
- 4 Pairwise distances and slopes



Open Source Routing Machine (OSRM):

- Production-grade routing engine
- Real street-level distances
- Fast table API for distance matrices
- OpenStreetMap data

Integration Pipeline:

- 1 Upload customer GPS coordinates
- 2 Query OSRM for distance matrix
- 3 Run DRL model inference
- 4 Reconstruct routes on map
- 5 Calculate real energy consumption

Scalability Strategy

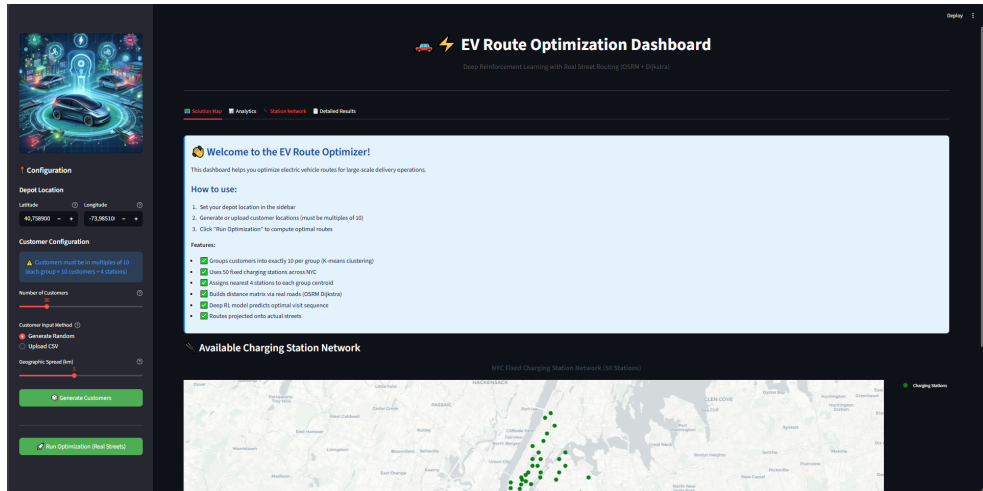
K-means Clustering:

- Group customers into batches
- 10 customers per group
- Run DRL on each group
- Combine results

NYC Configuration

50 pre-configured charging stations across New York City

Interactive Dashboard: Main Interface



The screenshot displays the 'EV Route Optimization Dashboard' interface. On the left is a dark sidebar with a 'Configuration' section containing input fields for 'Depot Location' (Latitude: 40.758900, Longitude: -73.98510), 'Customer Configuration' (a warning about group size), 'Number of Customers' (set to 30), 'Customer Input Method' (radio buttons for 'Generate Random' and 'Upload CSV'), and 'Geographic Spread (km)' (set to 5). At the bottom of the sidebar are two green buttons: 'Generate Customers' and 'Run Optimization (Real Streets)'. The main content area has a top header with a truck icon, a lightning bolt icon, and the title 'EV Route Optimization Dashboard', with a subtitle 'Deep Reinforcement Learning with Real Street Routing (OSRM + Dijkstra)'. Below the header is a navigation bar with tabs: 'Solution Map', 'Analytics', 'Station Network', and 'Detailed Results'. The 'Analytics' tab is active, showing a 'Welcome to the EV Route Optimizer!' message and a 'How to use:' section with three numbered steps. Below this is a 'Features:' section with a list of bullet points, each preceded by a green checkmark. At the bottom of the main area is a map titled 'Available Charging Station Network' and 'NYC Fixed Charging Station Network (30 Stations)', showing a map of New York City with green dots representing charging stations. A legend on the right indicates 'Charging Stations' with a green dot icon.

Welcome to the EV Route Optimizer!

This dashboard helps you optimize electric vehicle routes for large scale delivery operations.

How to use:

1. Set your depot location in the sidebar
2. Generate or upload customer locations (must be multiples of 30)
3. Click "Run Optimization" to compute optimal routes

Features:

- Groups customers into exactly 10 per group (N means clustering)
- Uses 50 fixed charging stations across NYC
- Assigns nearest 4 stations to each group centroid
- Builds distance matrix via real roads (OSRM Dijkstra)
- Deep RL model predicts optimal visit sequence
- Routes projected onto actual streets

Available Charging Station Network

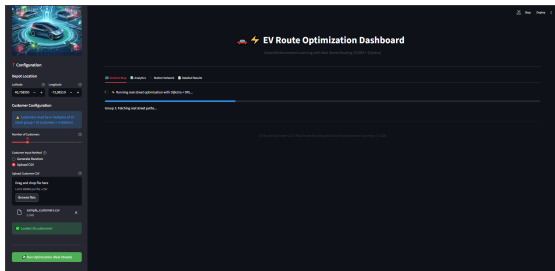
NYC Fixed Charging Station Network (30 Stations)



Charging Stations

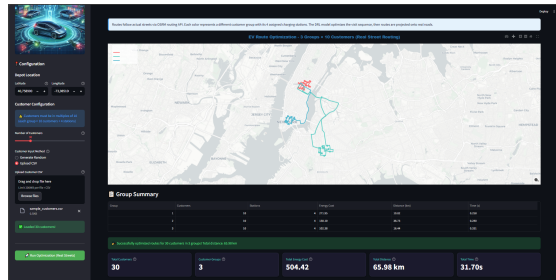
Features

Interactive Dashboard: Route Optimization



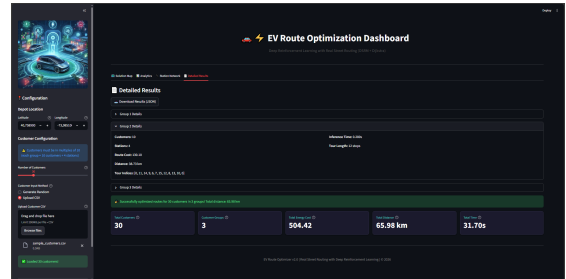
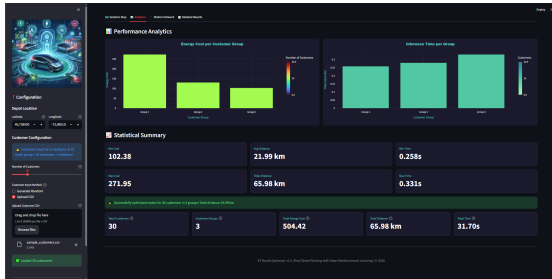
Inference Process

- Interactive map with customer locations and charging stations
- Real-time route visualization with street-level paths
- Color-coded routes for multiple vehicles



Optimized Routes

Interactive Dashboard: Analytics



Analytics Features

- Energy consumption breakdown per route segment
- SOC tracking throughout the journey
- Cost analysis and performance metrics
- Export results as JSON for integration

Training Configuration

Problem Configuration

Customers	10
Charging stations	4
Sequence length	16
Initial SOC	80%
Max load	50 units

Training Parameters

Epochs	20
Batch size	1,024
Training samples	1,280,000
Learning rate	10^{-4}
Baseline	Rollout

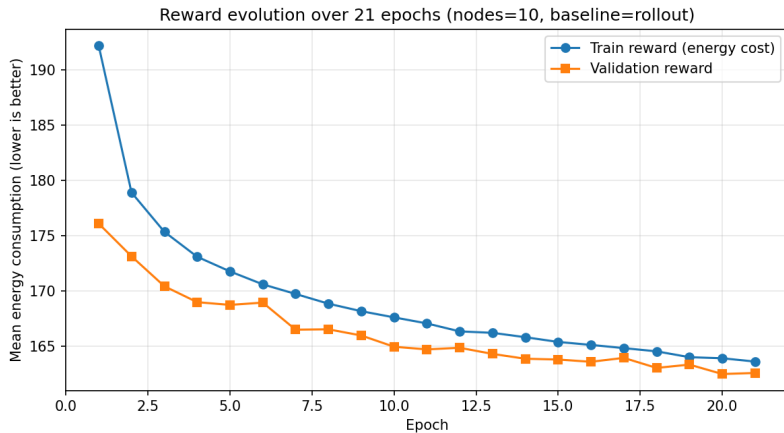
Model Architecture

Embedding dim	128
Encoder layers	3
Attention heads	8
FF hidden	512
Tanh clipping	10
Optimizer	Adam

Hardware

GPU	NVIDIA RTX
Training time	~8 hours

Training Results



Training Observations

Generalization

Performance Comparison

Method	Energy (kWh)	Inference Time
Google OR-Tools	12.45	2.1s
Hill Climbing	13.21	0.8s
Simulated Annealing	12.89	1.5s
Ant Colony Optimization	12.67	3.2s
DRL (Ours)	12.18	0.5s

Energy Reduction

- **2.2%** better than OR-Tools
- **7.8%** better than Hill Climbing
- Best overall energy efficiency

Speed Improvement

- **4x** faster than OR-Tools
- **6x** faster than ACO
- Real-time capable

Key Findings

Model Performance

- ✓ Generalizes to unseen instances
- ✓ Competitive solution quality
- ✓ Significantly faster inference
- ✓ Learns charging decisions

Energy Model Insights

- Load-dependent consumption
- Regenerative braking benefits
- Elevation-aware routing

Architecture Effectiveness

- Attention captures relationships
- Context embedding is crucial
- Masking ensures feasibility

Practical Benefits

- Ready for deployment
- Interactive visualization
- Real street routing
- Scalable to larger problems

Summary of Contributions

① Comprehensive MILP Formulation

- Physics-based energy consumption model
- Multi-trip capability with time windows
- Regenerative braking consideration

② Baseline Implementation

- OR-Tools, Hill Climbing, SA, ACO
- Standardized benchmarking framework

③ Deep Reinforcement Learning Solution

- Attention-based encoder-decoder
- EV-specific state representation
- Fast and generalizable

④ Production-Ready System

- Interactive Streamlit dashboard
- OSRM integration for real streets
- Export and analytics features

Limitations and Future Work

Current Limitations

- Trained on 10 customers
- Single depot scenario
- Homogeneous fleet
- Static demand
- No traffic dynamics

Future Directions

- Scale to 50+ customers
- Multi-depot extension
- Heterogeneous EV types
- Dynamic reoptimization
- Real-time traffic integration
- Joint routing + charging scheduling

Research Extensions

- Transfer learning for larger instances
- Multi-agent coordination for fleet management
- Integration with smart grid for optimal charging times

Deep Reinforcement Learning effectively addresses the Energy-Minimizing Electric Vehicle Routing Problem

Achieving competitive solution quality with significantly faster inference for real-world deployment



Learning-Based

Generalizes to new instances



Energy-Efficient

Physics-aware optimization



Production-Ready

Interactive dashboard

Thank You!

Questions & Discussion

Authors: Salem Fradi & Dhia Elhak Ezzeddini

Supervisor: Dr. Ferdaoues Chaabane

Institution: Sup'Com, Carthage University

Company: Sagemcom

Backup: Node Indexing Convention

Sequence Layout (10 customers, 4 stations)

Index	Node Type
0	Depot
1	Depot charging (near depot)
2–5	Charging stations (4 stations)
6–15	Customers (10 customers)

Total sequence length: $2 + 4 + 10 = 16$ nodes

Dynamic state tensor: 4 channels

- Load: remaining capacity
- Demand: per-node remaining demand
- SOC: battery state of charge
- Time: remaining route time budget

Feasibility conditions applied sequentially:

- 1 Base feasibility: demand > 0 AND demand $<$ current load
- 2 Previously selected node is masked
- 3 Charging access restrictions based on current location
- 4 Force depot return if no load OR no remaining demand
- 5 Block nodes unreachable due to SOC or time
- 6 Block if two-leg path to depot infeasible
- 7 Ensure depot always selectable when all customers masked

Purpose

Ensures energy, time, load, and demand constraints while permitting charging detours and driving termination

-  W. Kool, H. van Hoof, M. Welling, *“Attention, Learn to Solve Routing Problems!”*, ICLR, 2019.
-  Project OSRM - Open Source Routing Machine. <http://project-osrm.org/>
-  Google OR-Tools. <https://developers.google.com/optimization>
-  CVRPLIB: Capacitated Vehicle Routing Problem Library.
<http://vrp.atd-lab.inf.puc-rio.br/>
-  A. Vaswani et al., *“Attention Is All You Need”*, NeurIPS, 2017.